Radix Sort - Data Structures and Algorithms Tutorials

Last Updated : 23 Jul, 2025

Radix Sort is a linear sorting algorithm that sorts elements by processing them digit by digit. It is an efficient sorting algorithm for integers or strings with fixed-size keys.

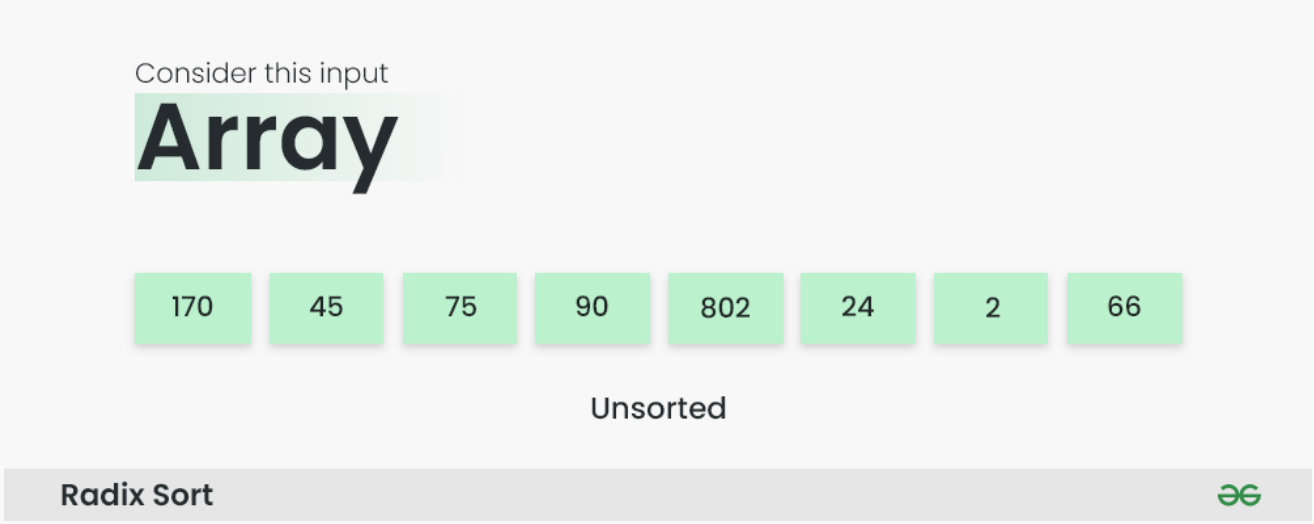
Rather than comparing elements directly, Radix Sort distributes the elements into buckets based on each digit's value. By repeatedly sorting the elements by their significant digits, from the least significant to the most significant, Radix Sort achieves the final sorted order.

Radix Sort Algorithm

The key idea behind Radix Sort is to exploit the concept of place value. It assumes that sorting numbers digit by digit will eventually result in a fully sorted list. Radix Sort can be performed using different variations, such as Least Significant Digit (LSD) Radix Sort or Most Significant Digit (MSD) Radix Sort.

How does Radix Sort Algorithm work?

To perform radix sort on the array [170, 45, 75, 90, 802, 24, 2, 66], we follow these steps:



How does Radix Sort Algorithm work | Step 1

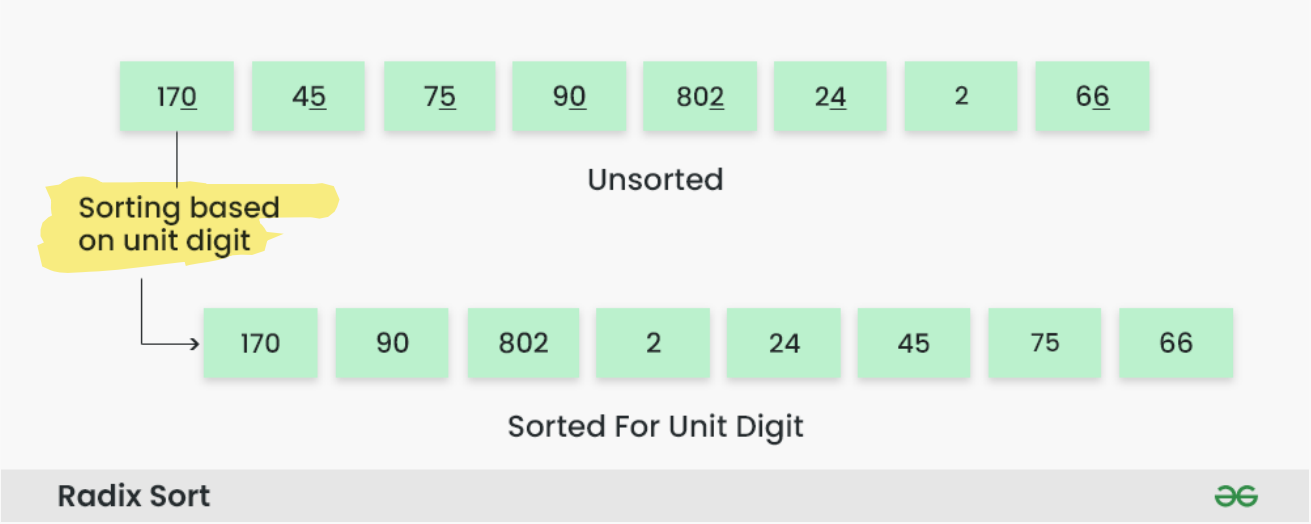
Step 1: *Find the largest element in the array, which is 802. It has three digits, so we will iterate three times, once for each significant place.*

Step 2: *Sort the elements based on the unit place digits (X=0). We use a stable sorting technique, such as counting sort, to sort the digits at each*

significant place. It's important to understand that the default implementation of counting sort is unstable i.e. same keys can be in a different order than the input array. To solve this problem, We can iterate the input array in reverse order to build the output array. This strategy helps us to keep the same keys in the same order as they appear in the input array.

Sorting based on the unit place:

- *Perform counting sort on the array based on the unit place digits.*
- *The sorted array based on the unit place is [170, 90, 802, 2, 24, 45, 75, 66].*

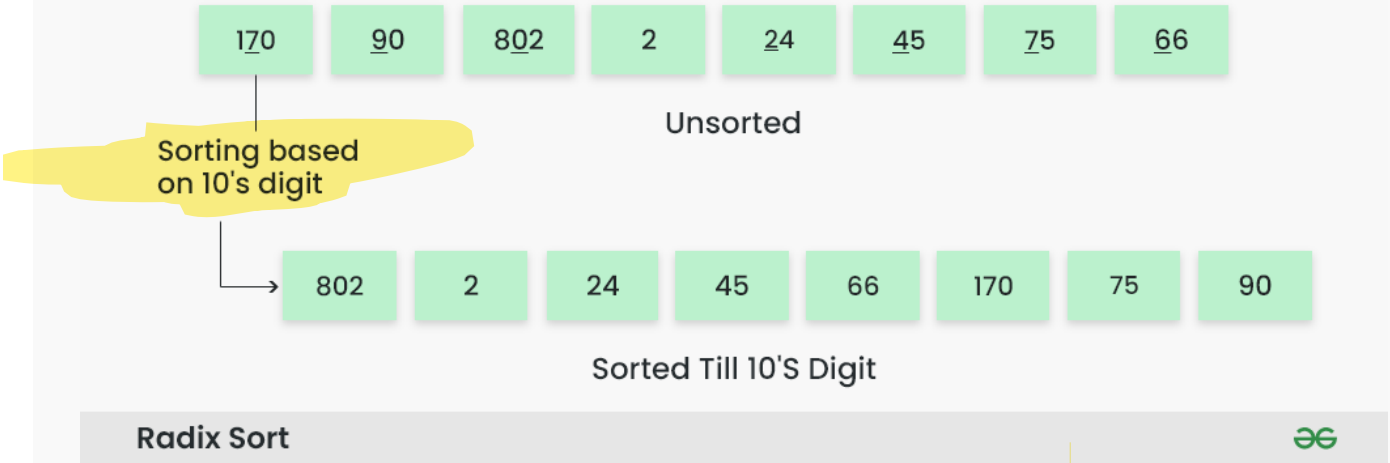


How does Radix Sort Algorithm work | Step 2

Step 3: *Sort the elements based on the tens place digits.*

Sorting based on the tens place:

- *Perform counting sort on the array based on the tens place digits.*
- *The sorted array based on the tens place is [802, 2, 24, 45, 66, 170, 75, 90].*

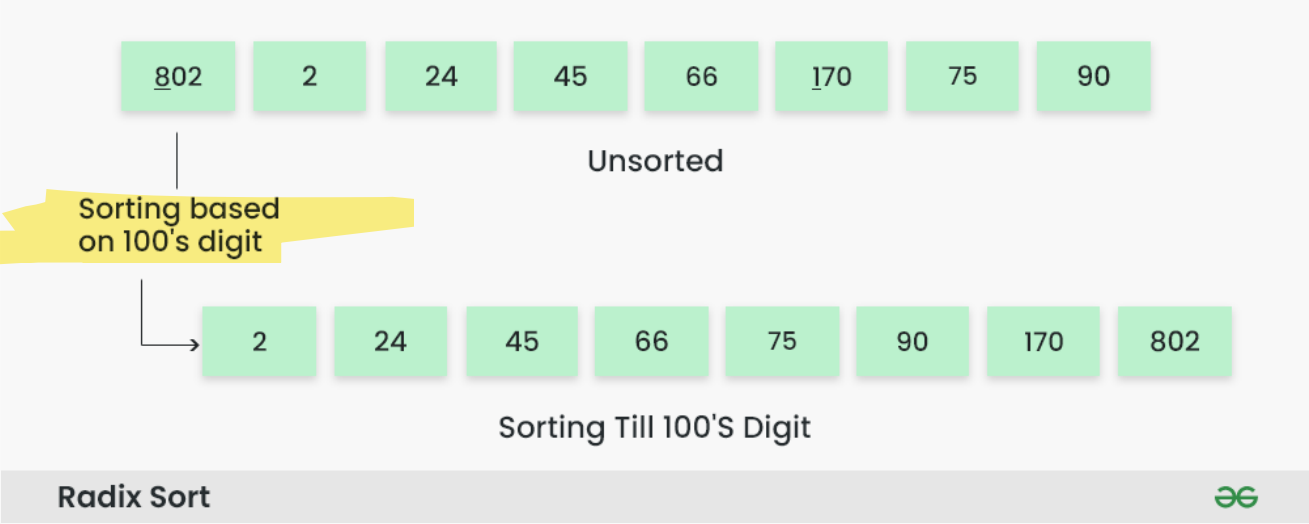


How does Radix Sort Algorithm work | Step 3

Step 4: *Sort the elements based on the hundreds place digits.*

Sorting based on the hundreds place:

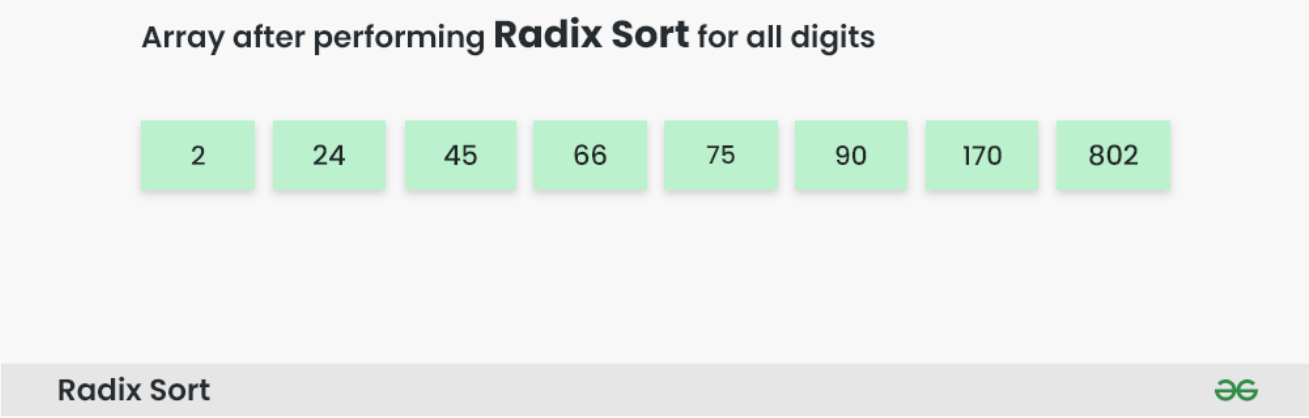
- *Perform counting sort on the array based on the hundreds place digits.*
- *The sorted array based on the hundreds place is [2, 24, 45, 66, 75, 90, 170, 802].*



How does Radix Sort Algorithm work | Step 4

Step 5: *The array is now sorted in ascending order.*

The final sorted array using radix sort is [2, 24, 45, 66, 75, 90, 170, 802].



How does Radix Sort Algorithm work | Step 5

Below is the implementation for the above illustrations:

C++ C Java Python C# JavaScript PHP Dart

```
// C++ implementation of Radix Sort
```

```cpp
#include <iostream>
using namespace std;

// A utility function to get maximum
// value in arr[]
int getMax(int arr[], int n)
{
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// A function to do counting sort of arr[]
// according to the digit
// represented by exp.
void countSort(int arr[], int n, int exp)
{

    // Output array
    int output[n];
    int i, count[10] = { 0 };

    // Store count of occurrences
    // in count[]
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Change count[i] so that count[i]
    // now contains actual position
    // of this digit in output[]
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
```

```
    // Copy the output array to arr[],
    // so that arr[] now contains sorted
    // numbers according to current digit
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}

// The main function to that sorts arr[]
// of size n using Radix Sort
void radixsort(int arr[], int n)
{

    // Find the maximum number to
    // know number of digits
    int m = getMax(arr, n);

    // Do counting sort for every digit.
    // Note that instead of passing digit
    // number, exp is passed. exp is 10^i
    // where i is current digit number
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

// A utility function to print an array
void print(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// Driver Code
int main()
{
    int arr[] = { 170, 45, 75, 90, 802, 24, 2, 66 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function Call
    radixsort(arr, n);
    print(arr, n);
    return 0;
}
```

Output

```
2 24 45 66 75 90 170 802
```

Complexity Analysis of Radix Sort:

Time Complexity:

- Radix sort is a non-comparative integer sorting algorithm that sorts data with integer keys by grouping the keys by the individual digits which share the same significant position and value. It has a time complexity of **O(d * (n + b))**, where d is the number of digits, n is the number of elements, and b is the base of the number system being used.
- In practical implementations, radix sort is often faster than other comparison-based sorting algorithms, such as quicksort or merge sort, for large datasets, especially when the keys have many digits. However, its time complexity grows linearly with the number of digits, and so it is not as efficient for small datasets.

Auxiliary Space:

- Radix sort also has a space complexity of **O(n + b),** where n is the number of elements and b is the base of the number system. This space complexity comes from the need to create buckets for each digit value and to copy the elements back to the original array after each digit has been sorted.

- Applications, Advantages and Disadvantages of Radix Sort

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem